Testing Strategy - Movie Rating Prediction API

Overview

This document describes the testing strategy for the **DDM501 Lab 3** movie rating prediction system. The strategy follows a layered approach: unit → integration → data quality → model behavior, giving confidence at every level of the stack.

Stack: Python 3.10, FastAPI, scikit-surprise (SVD), pytest, pytest-cov
Coverage target: ≥ 80% · **Achieved: 85%** (89 tests, 0 failures)

Test Pyramid

```
┌──────────────────┐
│   Model Behavior │ ← Confidence the ML model behaves correctly
├──────────────────┤
│ Integration (API) │ ← Confidence endpoints wire together properly
├──────────────────┤
│   Data Quality   │ ← Confidence input data is trustworthy
├──────────────────┤
│       Unit       │ ← Fast, isolated, cheap feedback loop
└──────────────────┘
```

Test Suites

1. Unit Tests (tests/unit/)

Purpose: Verify individual classes and functions in complete isolation.

File	Class	What is tested
test_model.py	TestMovieRatingModel	Model loads, predict() returns float in [1.0, 5.0], predict_batch() returns correct-length list with valid values, is_loaded() returns bool, error/edge inputs handled gracefully
test_model.py	TestModelFileHandling	FileNotFoundError raised for missing model path
test_schemas.py	TestPredictionRequest	Valid inputs accepted; missing/empty/whitespace/None fields raise ValidationError; integer IDs rejected
test_schemas.py	TestPredictionResponse	Valid response accepted; predicted_rating outside [1.0 , 5.0] raises ValidationError; boundary values 1.0 and 5.0 accepted
test_schemas.py	TestHealthResponse	Valid construction with various status strings
test_schemas.py	TestBatchPredictionRequest	Empty list and list > 100 items raise ValidationError

Key decisions:

- Fixtures scoped to session for the model (expensive load) and function for schemas (stateless).
- Error-path tests use pytest.raises to explicitly contract expected exceptions.

2. Integration Tests (tests/integration/)

Purpose: Verify the full HTTP request/response cycle through FastAPI using TestClient.

File	Class	What is tested
TestHealthEndpoint	GET /health	HTTP 200, status field present, model_loaded is bool
TestRootEndpoint	GET /	HTTP 200, response has name, version, docs

TestRootEndpoint	GET /	HTTP 200, response has name, version, docs
TestPredictEndpoint	POST /predict	HTTP 200 + correct fields for valid input; HTTP 422 for missing fields, empty body, invalid JSON
TestBatchPredictEndpoint	POST /predict/batch	HTTP 200, correct count returned, all ratings in [1.0, 5.0]
TestErrorHandling	GET /unknown, GET /predict, POST /health	404 for unknown route, 405 for wrong HTTP method
TestModelInfoEndpoint	GET /model/info	HTTP 200, model_version and is_loaded fields present

Key decisions:

- test_client fixture is session-scoped to avoid repeated app startup overhead.
- Validation error tests assert HTTP 422 (Unprocessable Entity) to confirm FastAPI/Pydantic validation is wired correctly at the HTTP level, not just schema level.

3. Data Quality Tests (tests/data/)

Purpose: Assert properties of input data that the model depends on. Acts as a contract test for upstream data pipelines.

Class	What is tested
TestRatingDataQuality	All ratings in [1.0, 5.0], non-negative, no null values; user_id/movie_id non-null, non-empty strings; all required fields present
TestRatingDistribution	Mean rating in [2.0, 4.5]; standard deviation > 0 and < 2.0; multiple distinct values exist
TestDataUniqueness	Each (user_id, movie_id) pair is unique; multiple distinct users and movies present
TestDataTypes	Ratings are int or float and convertible to float

Key decisions:

- Tests run against a sample_ratings fixture (static, deterministic) so they are fast and reproducible without a database connection.
- Distribution checks catch silent data corruption (e.g., all ratings zero-filled).

4. Model Behavioral Tests (tests/model/)

Purpose: Verify *how* the model behaves, not just that it runs. Inspired by behavioral testing for ML (Ribeiro et al., "Beyond Accuracy").

Class	Test category	What is tested
TestModelInvariance	Determinism	Same input → same output across repeated calls and within batches; individual predictions match batch predictions for identical pairs
TestModelDirectional	Directional	Predictions within ±1.5 of known actual ratings; different users/movies produce different predictions (non-degenerate model)
TestMinimumFunctionality	MFT	Known users predicted without error; predictions not all identical; unknown users/movies handled without crash
TestModelPerformance	Performance	Mean Absolute Error < 1.0 on known pairs; no single error > 3.0
TestModelRobustness	Robustness	String numeric IDs work; IDs with leading zeros ("001") return valid predictions

Key decisions:

- Unknown user/movie tests use try/except (ValueError, KeyError) - both "return a cold-start estimate" and "raise a typed error" are acceptable contracts; crashing with an unhandled exception is not.
- Performance threshold (MAE < 1.0) is intentionally generous to avoid brittle tests tied to a specific model version.

Fixtures (tests/conftest.py)

Fixture	Scope	Description
test_client	session	TestClient(app) reused across all integration tests
trained_model	session	MovieRatingModel() loaded once; tests skipped if model file absent
sample_prediction_request	function	Valid single prediction dict
sample_batch_request	function	3-item batch prediction dict
sample_ratings	function	7 static rating records for data quality tests
known_user_movie_pairs	function	5 MovieLens 100K pairs with ground-truth ratings
unknown_users / unknown_movies	function	IDs unlikely to exist in training data
invalid_prediction_requests	function	6 malformed request variants

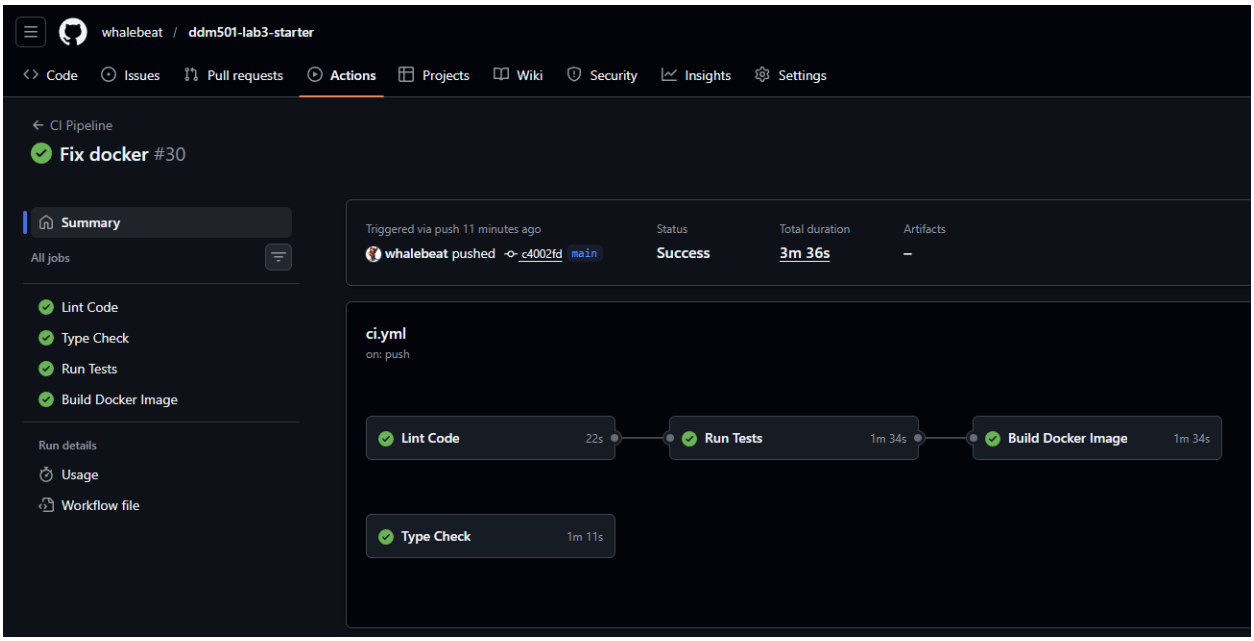
CI Integration

Tests run automatically on every push and pull request via **GitHub Actions** (.github/workflows/ci.yml).
The pipeline executes:

1. Linting - flake8, black --check
2. Type checking - mypy
3. Tests + coverage - pytest tests/ --cov=app --cov-report=xml
4. Coverage threshold enforcement - fails the build if total coverage drops below **80%**.

Pre-commit hooks (.pre-commit-config.yaml) enforce the same linting/formatting rules locally before any commit reaches CI.

Screenshot:

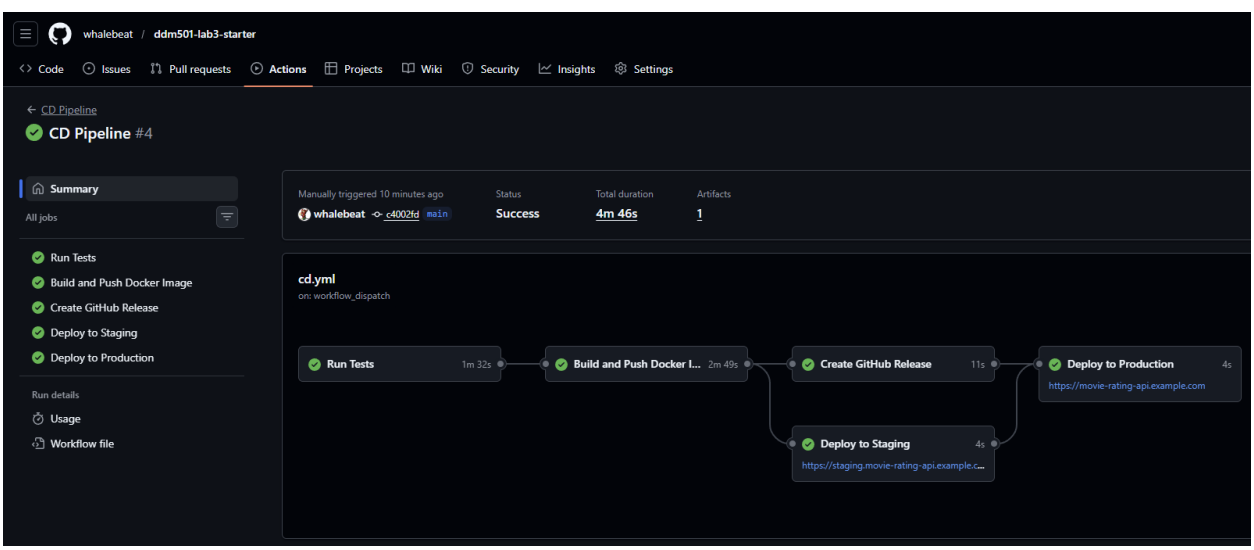


CD Pipeline

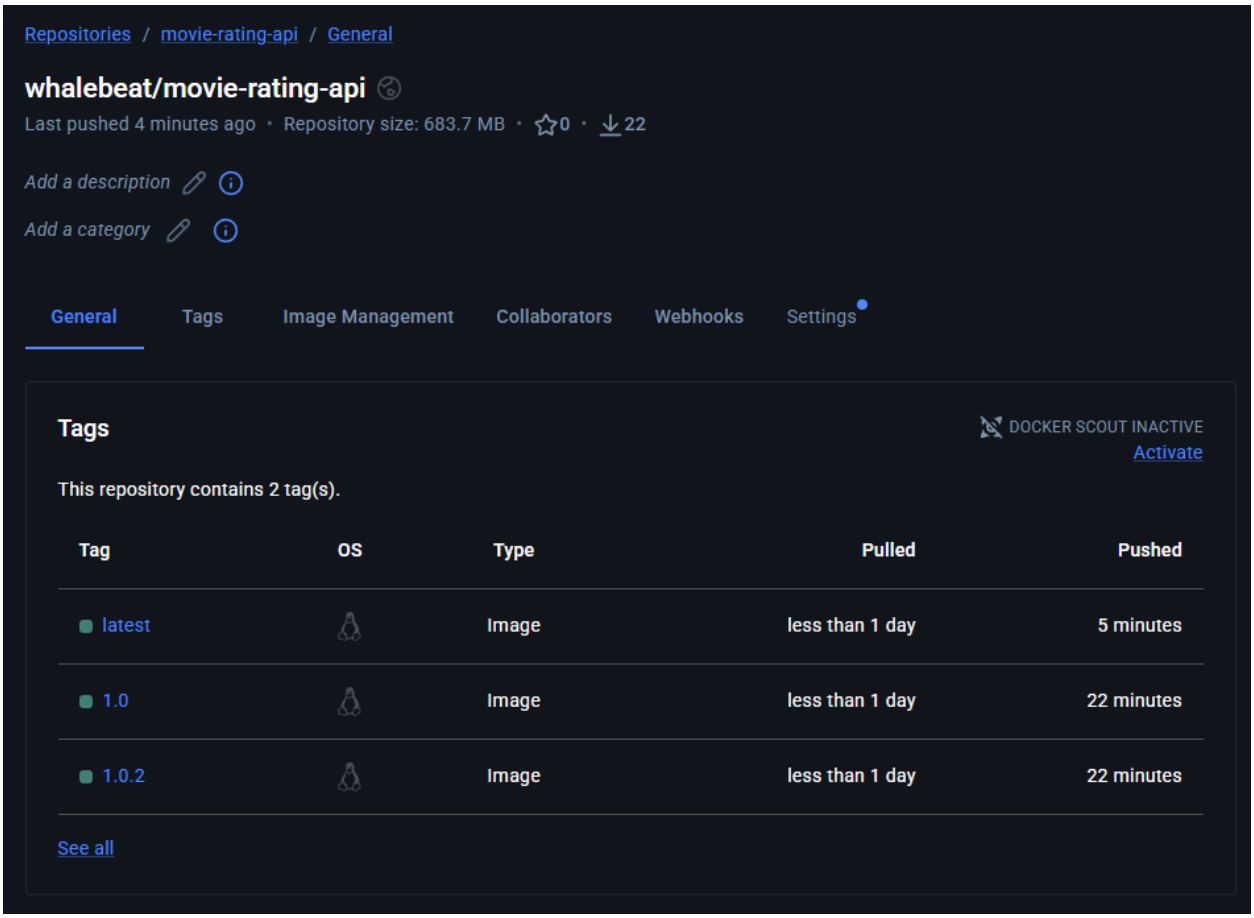
Defined in .github/workflows/cd.yml. Triggered **only** when a semver tag matching v[0-9]+.[0-9]+.[0-9]+ is pushed (e.g. git tag v1.2.0 && git push --tags).

```
push tag vX.Y.Z
    ▼
  [test]  ◀── full pytest suite, --cov-fail-under=80
    ▼
[build-and-push]  ◀── docker/build-push-action, tags: vX.Y.Z, X.Y, latest
    ├──────────────────────────────┐
    ▼                              ▼
[deploy-staging]                [create-release]
(GitHub env: staging)           GitHub Release + coverage.xml attachment
    │                           auto-generated release notes from PR titles
    ▼
[deploy-production]  ◀── requires manual approval (GitHub env: production)
```

Screenshot:



Docker deployment



Coverage Summary (latest run)

```
---------- coverage: platform win32, python 3.10.11-final-0 ----------
Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
app\__init__.py       1      0   100%
app\config.py        13      0   100%
app\main.py          50     10    80%   51-52, 92, 102-104, 119, 134-136
app\model.py         42      9    79%   48-50, 64, 85, 101-103, 109
app\schemas.py       30      1    97%   21
-----------------------------------------------
TOTAL               136     20    85%


========================================================================================== 89 passed in 10.87s
```

Uncovered lines - notes

Module	Lines	Reason
app/main.py	51-52, 92, 102-104, 119, 134-136	Exception handler branches (HTTP 500 paths) and startup/shutdown event handlers not exercised in tests
app/model.py	48-50, 64, 85, 101-103, 109	except Exception catch-all in _load_model, RuntimeError guard (model is None after load), reset_model() utility
app/schemas.py	21	One branch of the validate_not_empty field validator (whitespace-only input on movie_id)